

Technology Stack Recommendations

Based on existing familiarity with the MERN stack, MongoDB, and a Render (backend) + Vercel (frontend) + GitHub deployment setup.

Core Stack (keep this)

- **Frontend:** React (Vite, not CRA — faster builds/dev server) on Vercel
- **Backend:** Node.js + Express on Render
- **Database:** MongoDB Atlas (managed, not self-hosted — free tier is fine to start)
- **Repo/CI:** GitHub → auto-deploy to Render + Vercel on push

Additions for Smoother Client-Server Data Transfer

1. TanStack Query (React Query) — frontend

The single biggest upgrade for this app. Instead of manual `useEffect` + `fetch` + loading state everywhere, it provides automatic caching, background refetching, and easy **polling** (`refetchInterval`) — useful for the staff kanban board so cards update without a manual refresh. It also handles race conditions and stale data better than raw `fetch`/`axios` calls.

2. Socket.io — real-time updates

Polling works, but real-time push is worth it in two places:

- Staff dashboard: new order appears instantly in the "New Order" column without refreshing.
- Customer Track Order page: status updates live (e.g., "Ready for pickup") without reloading.

Socket.io works fine across Render (backend) and Vercel (frontend) — needs CORS configured for the Vercel domain. Pairs well with React Query: use sockets to *invalidate* a query cache, and let React Query handle the refetch.

3. Zod — shared validation

Define data shapes (order, product, payment) once with Zod schemas. Use them for backend request validation (Express middleware) and frontend form validation (works well with `react-hook-form`). Helps prevent field-name mismatches between frontend and backend.

4. TypeScript (optional, but worth considering)

If building from scratch, TypeScript + Zod together enforce order/product/payment types

end-to-end. Pays off once the app has 3 roles and several data models, especially for a solo/primary developer.

Infra-Specific Notes for Render + Vercel

- **Render free tier cold starts:** a real problem for a real-time ordering app — if the backend spins down after inactivity, the first order after idle time hits a 30–50s delay. Either upgrade to a paid Render instance, or use a lightweight cron ping (e.g., cron-job.org hitting a `/health` endpoint every 10 min) to keep it warm.
- **Environment variables:** keep MongoDB URI, payment gateway keys, and JWT secret in Render's/Vercel's dashboard env settings — never committed.
- **Vercel preview deployments:** every PR gets its own preview URL automatically — useful for testing the admin/staff panels before merging.

Closing the Payment Verification Gap

PayMongo (Philippines-focused, supports GCash) has a REST API that allows server-side verification of a transaction instead of trusting user-submitted reference numbers. Plugs into the existing Express backend as just another route — no architecture change needed.

Optional, if Scope Grows

- **Cloudinary or Uploadthing** for admin product images (signed upload, resizing, CDN delivery) rather than raw URL input — more reliable than asking admins to paste image links.
- **Twilio** or a local SMS API for pickup-ready notifications, since email open rates are poor for this use case.